

Formalização da correção do algoritmo Bubble Sort

Aquila Macedo Costa – 202021800
Douglas Nunes da Costa – 200016997
Rafael Oliveira Bonach – 221008365

13 de julho de 2026

1 Introdução e objetivo

Este trabalho formaliza a correção do Bubble Sort para listas de números naturais em Rocq/Coq. O algoritmo fazia parte das propostas do projeto e era adequado para estudar correção formal, especialmente porque permite separar a prova em duas propriedades: o resultado deve estar ordenado e deve ser uma permutação da lista original.

O arquivo *src/bubble_sort.v* contém as funções, os lemas e o teorema final. O relatório em PDF é gerado a partir dos comentários deste arquivo usando *coqdoc*, e o arquivo *bubblesort.hs* é produzido pela extração para Haskell.

2 Estratégia geral da prova

A correção foi dividida em preservação dos elementos e ordenação. Para a preservação dos elementos usamos *Permutation*. Para a ordenação usamos *Sorted le*, ou seja, listas ordenadas pela relação menor ou igual. No final, o teorema *bs_correto* junta essas duas partes.

3 Funcionamento de *bubble* e *bs*

A função *bubble* recebe uma lista de naturais e realiza uma única passagem de borbulhamento. Ela compara elementos consecutivos: se $x \leq y$, mantém x antes de y ; caso contrário, troca os dois elementos e continua a passagem.

Essa função é recursiva, mas não é estruturalmente recursiva no formato aceito diretamente por *Fixpoint*. No ramo em que ocorre troca, a chamada recursiva é feita sobre $x :: l$, que não aparece como subtermo direto de $x :: y :: l$. Por isso, a formalização usa *Function* com a medida *length l* para justificar a terminação.

Uma aplicação de *bubble* não ordena necessariamente uma lista arbitrária. Por exemplo, *bubble* (3 :: 2 :: 1 :: nil) produz 2 :: 1 :: 3 :: nil: os elementos são preservados e um elemento maior anda para a direita, mas a lista ainda não está ordenada.

A função principal *bs* ordena a cauda recursivamente e depois aplica *bubble* à lista formada pela cabeça original e pela cauda já processada. A prova de ordenação usa essa ideia: inserir um elemento no início de uma lista ordenada e aplicar uma passagem de *bubble* devolve uma lista ordenada.

4 Prova de preservação dos elementos/permutação

4.1 Permutação produzida por `bubble` (`bubble_perm`)

O lema `bubble_perm` afirma que uma passagem de `bubble` gera uma permutação da lista original. Ele é necessário porque `bs` usa `bubble` em cada passo; portanto, antes de provar que a lista final está ordenada, precisamos saber que uma passagem de borbulhamento não perde nem cria elementos.

Lemma `bubble_perm`: $\forall l, \text{Permutation } l \text{ (bubble } l \text{)}.$

A prova segue a própria definição recursiva de `bubble`, usando *functional induction*. Essa indução gera quatro casos.

No caso da lista vazia, `bubble nil` é `nil`. No caso de uma lista com um elemento, `bubble (x :: nil)` também devolve `x :: nil`. Esses dois casos são fechados por reflexividade da permutação, pois a função não altera a lista.

Quando a lista tem pelo menos dois elementos, a prova acompanha a comparação $x \leq? y$. Se a comparação é verdadeira, `bubble` mantém `x` na cabeça e continua sobre `y :: l0`. A hipótese de indução informa que `y :: l0` é uma permutação de `bubble (y :: l0)`. Como a cabeça `x` não muda, `perm_skip` leva essa permutação para as listas completas.

No ramo em que $x \leq? y$ é falso, a função troca os dois primeiros elementos e passa a calcular `bubble (x :: l0)`. A lista intermediária usada na prova é `y :: x :: l0`. Primeiro, `perm_swap` mostra que `x :: y :: l0` é permutação de `y :: x :: l0`. Depois, `perm_skip` usa a hipótese de indução para mostrar que `y :: x :: l0` é permutação de `y :: bubble (x :: l0)`. Por fim, `perm_trans` junta a troca inicial com a permutação obtida na chamada recursiva.

4.2 Preservação dos elementos em `bs` (`bs_perm`)

O lema `bs_perm` leva a preservação de elementos para a função completa `bs`.

Lemma `bs_perm`: $\forall l, \text{Permutation } l \text{ (bs } l \text{)}.$

A prova de `bs_perm` é por indução estrutural na lista `l`. No caso base $l = \text{nil}$, a definição de `bs` também retorna `nil`, então a permutação é imediata.

No passo indutivo, a lista tem a forma `h :: tl`. A hipótese de indução diz que `tl` é permutação de `bs tl`. Aplicando `perm_skip`, obtemos a primeira etapa da prova: `h :: tl` é permutação de `h :: bs tl`, pois a cabeça `h` é preservada.

A segunda etapa usa `bubble_perm`. Como `bs (h :: tl)` se reduz a `bubble (h :: bs tl)`, esse lema relaciona `h :: bs tl` com `bubble (h :: bs tl)`. A regra `perm_trans` junta as duas etapas: primeiro substituímos a cauda por sua versão ordenada por `bs`, depois usamos que a passagem de `bubble` não altera o multiconjunto de elementos.

5 Prova de ordenação

Para a parte de ordenação usamos `Sorted le`. A relação central é `le`, isto é, menor ou igual. Na definição utilizada pela biblioteca, `HdRel le h tl` relaciona a cabeça `h` com o primeiro elemento da cauda `tl`, quando ela não é vazia. Essa separação entre a cauda estar ordenada e a cabeça respeitar o primeiro elemento aparece várias vezes nas provas abaixo.

5.1 Preservação da ordenação por `bubble` (`bubble_stability`)

O lema `bubble_stability` mostra que uma lista já ordenada é um ponto fixo de `bubble`. O nome do lema não deve ser lido como estabilidade no sentido tradicional de algoritmos de ordenação; aqui ele afirma apenas que uma passagem de `bubble` não modifica uma lista que já satisfaz `Sorted le`.

Lemma `bubble_stability`: $\forall l, \text{Sorted } le \ l \rightarrow \text{bubble } l = l.$

A prova usa *functional induction* sobre *bubble*. Assim como em *bubble_perm*, aparecem os casos de lista vazia, lista unitária e listas com pelo menos dois elementos. Nos dois primeiros casos, *bubble* devolve a própria lista, então a igualdade é direta.

No ramo em que $x \leq y = \text{true}$, a função retorna $x :: \text{bubble } (y :: l)$. A hipótese *Sorted le* ($x :: y :: l$) é invertida: dela obtemos que a cauda $y :: l$ também está ordenada. Essa é exatamente a informação necessária para aplicar a hipótese de indução, que reescreve *bubble* ($y :: l$) como $y :: l$. Depois dessa reescrita, os dois lados da igualdade ficam iguais.

No ramo em que $x \leq y = \text{false}$, a comparação booleana é convertida em $x > y$. Esse caso só aparece na indução porque *bubble* precisa considerar todas as listas; ele é impossível sob a hipótese de que a entrada está ordenada. Ao inverter *Sorted le* ($x :: y :: l$), obtemos a relação $x \leq y$. Essa relação contradiz $x > y$, e a parte aritmética é fechada com *lia*.

5.2 Inserção da cabeça em uma cauda ordenada (*bubble_one*)

O lema *bubble_one* é o passo principal da ordenação: se a cauda l já está ordenada, então *bubble* ($x :: l$) também fica ordenada.

Lemma *bubble_one*: $\forall x l, \text{Sorted le } l \rightarrow \text{Sorted le } (\text{bubble } (x :: l))$.

A prova de *bubble_one* é a mais trabalhosa, porque ela mostra como uma passagem de *bubble* insere x em uma cauda que já está ordenada. A indução é estrutural sobre a lista l , que é a cauda recebida junto com x .

Caso base. Quando $l = \text{nil}$, *bubble* ($x :: \text{nil}$) resulta na lista unitária $x :: \text{nil}$. Para construir a prova de ordenação, usamos *Sorted_cons*: a cauda é ordenada por *Sorted_nil*, e a relação entre a cabeça e a cauda vazia é dada por *HdRel_nil*.

Passo indutivo. Quando a cauda tem a forma $h :: tl$, assumimos que ela está ordenada. A prova abre a equação de *bubble* ($x :: h :: tl$) e separa os casos da comparação $x \leq h$.

Se $x \leq h = \text{true}$, não ocorre troca entre x e h . O resultado tem a forma $x :: \text{bubble } (h :: tl)$. Pela hipótese da prova, $h :: tl$ já está ordenada. Com *bubble_stability*, podemos reescrever *bubble* ($h :: tl$) como $h :: tl$. A comparação booleana fornece $x \leq h$, que é exatamente a relação necessária para colocar x antes de h . Assim, a lista resultante permanece ordenada.

Se $x \leq h = \text{false}$, ocorre a troca inicial, e o resultado começa com h . Isso não é uma contradição: a comparação é convertida em $x > h$, de onde a prova usa a relação $h \leq x$ para manter h na cabeça. A partir daí, a prova precisa observar a forma de tl .

Quando $tl = \text{nil}$, a parte recursiva é *bubble* ($x :: \text{nil}$), ou seja, uma lista unitária. A ordenação de $h :: x :: \text{nil}$ é construída com *Sorted_cons*, usando que a lista unitária é ordenada e que $h \leq x$.

Quando $tl = n :: l$, a prova abre uma nova comparação, agora entre x e n . No ramo $x \leq n = \text{true}$, a chamada recursiva mantém x antes de n ; a hipótese de indução trata a ordenação dessa parte recursiva, e $h \leq x$ fornece a relação da cabeça h com o primeiro elemento da cauda resultante.

No ramo $x \leq n = \text{false}$, x também passa por n , e a chamada recursiva é tratada novamente pela hipótese de indução. A diferença é que agora o primeiro elemento depois de h é n . Como a lista original $h :: n :: l$ estava ordenada, a inversão dessa hipótese fornece a relação necessária $h \leq n$. Com isso, *Sorted_cons* fecha a ordenação mantendo h na cabeça.

5.3 Ordenação produzida por *bs* (*bs_sorted*)

O lema *bs_sorted* usa *bubble_one* para provar que *bs* sempre retorna uma lista ordenada.

Lemma *bs_sorted*: $\forall l, \text{Sorted le } (bs \ l)$.

A prova de *bs_sorted* é por indução estrutural na lista. No caso base, *bs nil* é *nil*, e a lista vazia é ordenada por *Sorted_nil*.

No passo indutivo, a lista é $h :: tl$. Pela definição da função, *bs* ($h :: tl$) é *bubble* ($h :: bs \ tl$). A hipótese de indução garante que *bs tl* está ordenada. Portanto podemos aplicar *bubble_one* com $x = h$ e $l = bs \ tl$. O lema conclui exatamente que *bubble* ($h :: bs \ tl$) está ordenada, que é o resultado de *bs* nesse caso.

5.4 Correção final do algoritmo (`bs_correto`)

O teorema `bs_correto` junta as duas propriedades necessárias para a correção: ordenação e preservação dos elementos.

Theorem `bs_correto`: $\forall l, \text{Sorted } le (bs\ l) \wedge \text{Permutation } l (bs\ l)$.

Essa prova não precisa de indução nova. O objetivo é uma conjunção: mostrar que a saída está ordenada e que preserva os elementos da entrada. Usamos `split` para separar as duas partes. A primeira é resolvida por `bs_sorted`, e a segunda por `bs_perm`. Juntas, essas propriedades mostram que `bs` retorna uma lista ordenada por `le` e com os mesmos elementos da lista original.

6 Extração para Haskell

O mecanismo de extração de Rocq/Coq gera o arquivo `bubblesort.hs` a partir da função formalizada. As provas são apagadas durante a extração, pois servem para a verificação dentro do assistente de provas. Assim, o arquivo Haskell resultante corresponde ao conteúdo computacional da função `bs`, mas não substitui a prova feita em Rocq/Coq.

7 Decisões e referências

O desenvolvimento usa `Sorted le` e `Permutation` da biblioteca padrão, além de `Function` com a medida `length` para definir `bubble`. Essa escolha foi necessária porque `bubble` é recursiva, mas não estruturalmente recursiva no formato direto de `Fixpoint`. O projeto partiu do enunciado e do repositório-base fornecidos pelo professor.

Ferramentas de IA foram utilizadas apenas como apoio na revisão da documentação; o grupo revisou o texto em relação às provas do arquivo.

8 Conclusão

A formalização mostra que `bs` retorna uma lista ordenada por `le` e com os mesmos elementos da entrada. Os lemas auxiliares deixam a prova mais clara: primeiro mostramos que `bubble` e `bs` preservam os elementos; depois usamos `bubble_one` para provar que inserir uma cabeça em uma cauda ordenada e aplicar `bubble` produz novamente uma lista ordenada; por fim, juntamos essas propriedades no teorema `bs_correto`.

Repositório: <https://github.com/RafaBonach/bubble-sort-lcl.git>